

# 理论计算机科学导引

Lecture Note

Huijiao

2026 年 8 月 6 日

# 目录

|                                           |           |
|-------------------------------------------|-----------|
| <b>第 1 章 引子</b>                           | <b>1</b>  |
| 1.1 二进制串与编码                               | 1         |
| 1.2 可数集合                                  | 3         |
| 1.3 问题的公理化定义                              | 4         |
| <b>第 2 章 布尔电路</b>                         | <b>5</b>  |
| 2.1 电路模型                                  | 5         |
| 2.2 AON-Circ 程序与 NAND 门                   | 5         |
| 2.3 任意布尔函数的电路实现                           | 6         |
| 2.3.1 NAND-Circ 程序中的语法糖 (syntactic sugar) | 7         |
| 2.3.2 ADD 与 LOOKUP                        | 7         |
| 2.3.3 精细上界                                | 9         |
| 2.3.4 计数下界                                | 10        |
| 2.4 程序编码与通用求值                             | 11        |
| <b>第 3 章 布尔函数与语言</b>                      | <b>13</b> |
| 3.1 从一般函数到布尔函数                            | 13        |
| 3.2 语言与布尔函数                               | 14        |
| <b>第 4 章 自动机</b>                          | <b>15</b> |
| 4.1 确定有限自动机                               | 15        |
| 4.2 正则语言的封闭性                              | 16        |
| 4.3 非确定有限自动机                              | 17        |
| 4.3.1 NFA 与 DFA 的等价性                      | 18        |
| 4.3.2 拼接与 Kleene 星                        | 18        |
| 4.4 正则表达式                                 | 19        |
| 4.5 正则语言的泵引理                              | 20        |
| <b>第 5 章 PDA 与语法</b>                      | <b>22</b> |
| 5.1 下推自动机                                 | 22        |

|              |                            |           |
|--------------|----------------------------|-----------|
| 5.2          | 上下文无关文法 . . . . .          | 24        |
| 5.3          | PDA 与 CFG 的等价性 . . . . .   | 25        |
| <b>第 6 章</b> | <b>图灵机</b>                 | <b>27</b> |
| 6.1          | 图灵机模型 . . . . .            | 27        |
| 6.2          | NAND-TM 编程语言 . . . . .     | 28        |
| 6.2.1        | NAND-TM 程序中的语法糖 . . . . .  | 29        |
| 6.3          | NAND-RAM . . . . .         | 29        |
| 6.4          | Church–Turing 论题 . . . . . | 30        |
| <b>第 7 章</b> | <b>可计算性</b>                | <b>31</b> |
| 7.1          | 通用图灵机 . . . . .            | 31        |
| 7.2          | 不可计算函数 . . . . .           | 32        |
| 7.3          | 停机问题 . . . . .             | 32        |
| 7.4          | 归约 . . . . .               | 33        |
| 7.5          | Rice 定理 . . . . .          | 34        |
| 7.6          | 自引用与递归定理 . . . . .         | 34        |
| 7.6.1        | 最小机器与不动点 . . . . .         | 36        |
| 7.7          | 有效证明系统与不完备性 . . . . .      | 37        |
| 7.7.1        | 量化整数语句 . . . . .           | 38        |

# 第 1 章 引子

理论计算机科学经常把两个层面分开讨论：函数 (function) 描述要计算什么，是问题的规格；程序 (program) 描述如何计算，是规格的一种实现。从函数到程序，就是把抽象的东西具体化。本讲先用二进制串 (binary string) 形式化“描述”，再比较函数与程序在数量上的差异。

全文约定

$$\mathbb{N} = \{0, 1, 2, \dots\}, \quad \Sigma = \{0, 1\}.$$

## 1.1 二进制串与编码

**定义 1.1** (二进制串). 对任意  $n \in \mathbb{N}$ , 定义

$$\Sigma^n = \{(a_1, \dots, a_n) : a_i \in \Sigma\}.$$

特别地,  $\Sigma^0 = \{\varepsilon\}$ , 其中  $\varepsilon$  表示空串 (empty string)。记

$$\Sigma^* = \bigcup_{n \geq 0} \Sigma^n, \quad \Sigma^+ = \Sigma^* \setminus \{\varepsilon\}.$$

对  $x \in \Sigma^n$ , 记其长度为  $|x| = n$ 。

对  $x, y \in \Sigma^*$ , 把二者的串联记为  $xy$ 。如果存在  $z \in \Sigma^*$  使得  $y = xz$ , 则称  $x$  是  $y$  的前缀 (prefix)。如果还能取到  $z \neq \varepsilon$ , 则称  $x$  是  $y$  的真前缀 (proper prefix)。

**定义 1.2** (编码). 集合  $A$  上的一个 (二进制) 编码 (encoding) 是单射 (injective map)

$$E : A \longrightarrow \Sigma^*.$$

这里, 单射是指: 对任意  $x, y \in A$ , 若  $E(x) = E(y)$ , 则  $x = y$ 。

**例 1.1.** 令  $b(n)$  为自然数  $n$  的标准二进制表示, 并尝试用

$$P(a, b) = b(a)b(b)$$

编码  $\mathbb{N} \times \mathbb{N}$ 。这个映射不是单射。例如

$$P(1, 6) = 1\ 110 = 1110 = 11\ 10 = P(3, 2).$$

问题在于直接串联后无法确定两个分量的分界位置。因此我们要考虑一种更优秀的编码, 来避免这个问题。

**定义 1.3** (无前缀编码). 编码  $E: A \rightarrow \Sigma^+$  称为无前缀的 (prefix-free), 如果对任意不同的  $x, y \in A$ , 码字  $E(x)$  都不是  $E(y)$  的前缀。

记  $A^*$  为由  $A$  中元素组成的所有有限序列的集合, 其空序列仍记为  $\varepsilon$ 。对映射  $E: A \rightarrow \Sigma^+$ , 定义其逐项串联扩张

$$\overline{E}(a_1, \dots, a_n) = \begin{cases} E(a_1) \cdots E(a_n), & n > 0, \\ \varepsilon, & n = 0. \end{cases} \quad (1.1)$$

这个定义很自然, 我们要说明这个自然定义确实是个单射。

**引理 1.1.** 若  $E: A \rightarrow \Sigma^+$  是无前缀编码, 则由 式 (1.1) 定义的  $\overline{E}: A^* \rightarrow \Sigma^*$  是单射。

证明. 假设两组序列的编码相同:

$$\overline{E}(a_1, \dots, a_p) = \overline{E}(b_1, \dots, b_q).$$

码字非空, 所以  $p = 0 \iff q = 0$ 。若  $p, q > 0$ , 则  $E(a_1)$  与  $E(b_1)$  中较短的码字一定是较长码字的前缀。由无前缀性可知

$$E(a_1) = E(b_1) \implies a_1 = b_1.$$

两边消去首个码字并归纳, 得到

$$p = q, \quad a_i = b_i \quad (1 \leq i \leq p).$$

故  $\overline{E}$  是单射。 □

**定理 1.1** (编码的无前缀化). 若存在编码  $E: A \rightarrow \Sigma^*$ , 则存在无前缀编码  $E_{\text{pf}}: A \rightarrow \Sigma^+$ , 并且对每个  $a \in A$  都有

$$|E_{\text{pf}}(a)| = 2|E(a)| + 2.$$

证明. 定义  $h(0) = 00$ 、 $h(1) = 11$ , 并令  $h$  逐位作用于二进制串。取

$$E_{\text{pf}}(a) = h(E(a))01, \quad |E_{\text{pf}}(a)| = 2|E(a)| + 2.$$

按两个比特分组时, 正文块属于  $\{00, 11\}$ , 终止块为  $01$ 。若  $E_{\text{pf}}(a)$  是  $E_{\text{pf}}(b)$  的前缀, 则短串的终止块必须与长串的某一块对齐。正文中没有  $01$ , 故它只能与长串的终止块对齐。于是

$$E_{\text{pf}}(a) = E_{\text{pf}}(b) \implies E(a) = E(b) \implies a = b.$$

因此  $E_{\text{pf}}$  无前缀。 □

思考题. 能否把长度开销改进到  $|E(a)| + O(\log|E(a)|)$ ?

解答. 可以. 令

$$n = |E(a)|, \quad \ell = |\text{bin}(n+1)|,$$

并定义

$$L(n) = 1^\ell 0 \text{ bin}(n+1), \quad E'(a) = L(n)E(a).$$

解码过程为

$$1^\ell 0 \longrightarrow \ell \longrightarrow \text{bin}(n+1) \longrightarrow n \longrightarrow E(a).$$

其中最后一步恰好读取  $n$  位. 并且

$$|E'(a)| = n + 2\lfloor \log_2(n+1) \rfloor + 3 = n + O(\log(n+1)).$$

$L(n)$  自定界; 当  $n$  相同时,  $E'(a)$  等长. 因此该构造无前缀. □

**推论 1.1.** 若存在编码  $E: A \rightarrow \Sigma^*$ , 则存在编码  $F: A^* \rightarrow \Sigma^*$ .

证明. 先用定理 1.1 把  $E$  无前缀化, 再用引理 1.1 逐项串联即可. □

## 1.2 可数集合

**定义 1.4** (至多可数). 如果存在单射  $f: A \rightarrow \mathbb{N}$ , 则称集合  $A$  是至多可数的 (at most countable). 换言之,  $A$  或者是有限集, 或者可以与自然数集  $\mathbb{N}$  建立双射 (bijection). 当  $A \neq \emptyset$  时, 这也等价于存在满射 (surjection)  $g: \mathbb{N} \rightarrow A$ .

注. 非空条件只用于满射表述, 因为不存在  $\mathbb{N} \rightarrow \emptyset$ . 有些文献把本文的“至多可数”简称为“可数”.

**引理 1.2.**  $\Sigma^*$  是可数无限集.

证明. 对  $x \in \Sigma^n$ , 令  $\text{val}_2(x)$  为其二进制数值 (允许前导零), 并定义

$$\nu(x) = 2^n - 1 + \text{val}_2(x).$$

于是

$$\nu(\Sigma^n) = \{2^n - 1, \dots, 2^{n+1} - 2\}, \quad \mathbb{N} = \bigsqcup_{n \geq 0} \nu(\Sigma^n).$$

故  $\nu: \Sigma^* \rightarrow \mathbb{N}$  是双射. □

**定理 1.2** (可编码性与可数性). 集合  $A$  至多可数, 当且仅当存在编码  $E: A \rightarrow \Sigma^*$ .

证明. 由引理 1.2, 存在双射  $\nu: \Sigma^* \rightarrow \mathbb{N}$ .

若  $E: A \rightarrow \Sigma^*$  是单射, 则

$$\nu \circ E: A \rightarrow \mathbb{N}$$

也是单射. 反之, 若  $f: A \rightarrow \mathbb{N}$  是单射, 令  $c(n) = 1^n 0$ , 则

$$c \circ f: A \rightarrow \Sigma^*$$

是一个编码. □

### 1.3 问题的公理化定义

因为我们已经有编码了，所以我们可以把问题公理化了。

**定义 1.5** (问题). 在本讲义中，一个 (计算) 问题 (computational problem) 是函数

$$f : \Sigma^* \longrightarrow \Sigma^*.$$

所有问题构成的集合记为

$$\mathcal{P} = (\Sigma^*)^{\Sigma^*}.$$

**定理 1.3.** 问题集合  $\mathcal{P}$  不可数。

证明 (*Cantor* 对角化). 由[引理 1.2](#), 可写

$$\Sigma^* = \{s_0, s_1, s_2, \dots\}.$$

反设  $\mathcal{P} = \{f_0, f_1, f_2, \dots\}$ 。定义

$$g(s_i) = \begin{cases} 0, & f_i(s_i) \neq 0, \\ 1, & f_i(s_i) = 0. \end{cases}$$

因而

$$\forall i \in \mathbb{N}, \quad g(s_i) \neq f_i(s_i) \implies g \neq f_i.$$

于是  $g \in \mathcal{P}$  却不在上述枚举中，矛盾。

□

## 第 2 章 布尔电路

布尔电路 (Boolean circuit) 用有限个逻辑门 (logic gate) 把输入比特 (bit) 变成输出比特。本讲使用三种基本逻辑门: 与门 (AND)、或门 (OR) 和非门 (NOT)。

### 2.1 电路模型

一个布尔电路  $C$  可以看成有限的有向无环图 (directed acyclic graph, DAG)。图中包含:

1.  $n$  个输入结点  $x_0, \dots, x_{n-1}$ ;
2. 若干内部结点, 每个结点标记为 AND、OR 或 NOT 门;
3.  $m$  个指定的输出结点  $y_0, \dots, y_{m-1}$ 。

AND 和 OR 门各有两个输入, NOT 门有一个输入。因为图中没有有向环, 所以各个门可以按拓扑顺序 (topological order) 依次求值。电路的规模 (size) 是其中逻辑门的数量, 记为  $|C|$ 。

**定义 2.1** (电路计算). 设  $f: \{0, 1\}^n \rightarrow \{0, 1\}^m$ 。如果对每个输入  $x \in \{0, 1\}^n$ , 电路  $C$  的  $m$  个输出比特恰好组成  $f(x)$ , 则称  $C$  计算函数  $f$ 。

**例 2.1.** 异或函数 (exclusive OR, XOR) 可以写成

$$\text{XOR}(a, b) = \neg(a \wedge b) \wedge (a \vee b),$$

因而可以用 AND、OR 和 NOT 门组成的布尔电路计算。

### 2.2 AON-Circ 程序与 NAND 门

**定义 2.2** (AON-Circ 程序). AON-Circ 程序是一列按顺序执行的赋值语句 (assignment statement)。每条语句调用一次 AND、OR 或 NOT, 并且只能读取输入变量或此前语句已经算出的变量。程序最后指定  $m$  个变量作为输出。程序的规模是赋值语句的行数。



例如，一条语句可以具有以下三种形式之一：

$$z_i \leftarrow z_j \wedge z_k, \quad z_i \leftarrow z_j \vee z_k, \quad z_i \leftarrow \neg z_j,$$

其中右侧的变量必须已经定义。

**定理 2.1.** 函数  $f$  能由布尔电路计算，当且仅当它能由 *AON-Circ* 程序计算。两种表示可以保持规模不变：电路的门数等于对应程序的行数。

证明. 给定电路，按拓扑顺序为每个门写一条赋值语句，即得到 *AON-Circ* 程序。反过来，为程序的每一行建立一个门，并把该行读取的变量连接到这个门，即得到布尔电路。每个门与每行语句一一对应。□

与非门 (NAND) 定义为

$$\text{NAND}(a, b) = \neg(a \wedge b).$$

只使用 NAND 门便可实现三个基本门：

$$\begin{aligned} \neg a &= \text{NAND}(a, a), \\ a \wedge b &= \text{NAND}(\text{NAND}(a, b), \text{NAND}(a, b)), \\ a \vee b &= \text{NAND}(\text{NAND}(a, a), \text{NAND}(b, b)). \end{aligned}$$

因此 NAND 电路与普通布尔电路可以相互转换。若原电路规模为  $s$ ，则转换后分别满足

$$|C_{\text{AON}}| \leq 2|C_{\text{NAND}}|, \quad |C_{\text{NAND}}| \leq 3|C_{\text{AON}}|.$$

## 2.3 任意布尔函数的电路实现

**定理 2.2.** 对任意  $n, m \geq 1$  以及任意函数

$$f : \{0, 1\}^n \longrightarrow \{0, 1\}^m,$$

都存在计算  $f$  的布尔电路。进一步，电路规模可以达到

$$|C| = O\left(\frac{m2^n}{n}\right).$$

**粗略上界的构造.** 先考虑  $f$  的第  $j$  个输出比特  $f_j$ 。对每个  $a = (a_0, \dots, a_{n-1}) \in \{0, 1\}^n$ ，定义

$$T_a(x) = \bigwedge_{r=0}^{n-1} \ell_{a,r}(x_r), \quad \ell_{a,r}(x_r) = \begin{cases} x_r, & a_r = 1, \\ \neg x_r, & a_r = 0. \end{cases}$$

当且仅当  $x = a$  时,  $T_a(x) = 1$ 。因此

$$f_j(x) = \bigvee_{\substack{a \in \{0,1\}^n \\ f_j(a)=1}} T_a(x).$$

这就是析取范式 (disjunctive normal form, DNF) 的直接构造。每个  $T_a$  使用  $O(n)$  个门, 而这样的项至多有  $2^n$  个, 所以一个输出比特需要  $O(n2^n)$  个门。分别构造  $m$  个输出, 便得到

$$|C| = O(mn2^n).$$

定理 2.2 中的  $O(m2^n/n)$  上界去掉了这个直接构造中的冗余。这个更精细的上界将在后文证明。

**定义 2.3** (通用门组). 如果只使用门集合  $\mathcal{F}$  中的门就能计算任意布尔函数, 则称  $\mathcal{F}$  是通用的 (universal)。

由定理 2.2 和上面的三个恒等式, NAND 门本身就是通用的。因此, 要判断一组门  $\mathcal{F}$  是否通用, 只需判断能否仅用  $\mathcal{F}$  中的门实现 NAND 函数。

为了证明定理 2.2, 我们先来介绍一些特性和例子。

### 2.3.1 NAND-Circ 程序中的语法糖 (syntactic sugar)

为了让程序更容易书写, 可以暂时加入以下三种语法糖:

1. **定长循环 (fixed-length loop)**: 把循环体复制固定次数;
2. **用户定义过程 (user-defined procedure)**: 在调用处展开过程体;
3. **条件语句 (conditional statement)**: 同时计算两个分支, 再用选择器 (multiplexer, MUX) 输出其中一个。

条件语句中的一比特选择器可以写成

$$\text{MUX}(b, u, v) = (\neg b \wedge u) \vee (b \wedge v),$$

其中  $b = 0$  时输出  $u$ ,  $b = 1$  时输出  $v$ 。循环次数和过程参数一旦固定, 上述写法都能展开成普通的 NAND-Circ 程序, 因此不会增强计算能力。

### 2.3.2 ADD 与 LOOKUP

**例 2.2** (ADD). 输入包含两个  $n$  位二进制数。按从低位到高位顺序, 将它们分别记为

$$(x_0, \dots, x_{n-1}), \quad (x_n, \dots, x_{2n-1})$$

加法函数为

$$\text{ADD}_n : \{0, 1\}^{2n} \longrightarrow \{0, 1\}^{n+1}.$$

令  $\text{MAJ}(a, b, c)$  表示三个比特的多数值 (majority), 即

$$\text{MAJ}(a, b, c) = (a \wedge b) \vee (a \wedge c) \vee (b \wedge c).$$

可以用如下定长循环实现逐位进位加法:

```
result = [0] * (n + 1)
carry = [0] * (n + 1)
for i in range(n):
    result[i] = XOR(carry[i], XOR(x[i], x[i+n]))
    carry[i+1] = MAJ(carry[i], x[i], x[i+n])
result[n] = carry[n]
return result
```

XOR 和 MAJ 都能用常数个 NAND 门实现。把循环展开后, 程序共有  $O(n)$  行, 因而加法电路的规模为  $O(n)$ 。

**例 2.3** (LOOKUP). 输入由  $2^k$  个数据比特  $x_0, \dots, x_{2^k-1}$  和  $k$  个索引比特  $i_0, \dots, i_{k-1}$  组成。令

$$j = \sum_{r=0}^{k-1} i_r 2^{k-1-r}.$$

查表函数 (lookup function) 返回第  $j$  个数据比特:

$$\text{LOOKUP}_k : \{0, 1\}^{2^k+k} \longrightarrow \{0, 1\}, \quad \text{LOOKUP}_k(x, i) = x_j.$$

它可以递归地把数据表分成两半:

```
lookup(x, index):
    if len(index) == 1:
        if index[0] == 0: return x[0]
        else: return x[1]
    half = len(x) / 2
    if index[0] == 0:
        return lookup(x[0:half], index[1:])
    else:
        return lookup(x[half:], index[1:])
```

对电路而言, 条件语句的两个分支都要构造, 再由 MUX 选择输出。若  $L(k)$  表示展开后的程序行数, 则

$$L(k) = 2L(k-1) + O(1), \quad L(1) = O(1).$$

因此  $L(k) = O(2^k)$ , LOOKUP 电路的规模也是  $O(2^k)$ 。

### 2.3.3 精细上界

**定理 2.2** 的精细上界证明. 先考虑单输出函数  $f: \{0, 1\}^n \rightarrow \{0, 1\}$ . 取  $k < n$ , 并把输入写成

$$x = (u, v), \quad u \in \{0, 1\}^k, \quad v \in \{0, 1\}^{n-k}.$$

对每个  $v \in \{0, 1\}^{n-k}$ , 定义  $k$  元函数

$$h_v(u) = f(u, v).$$

先共享地构造全部  $2^k$  个最小项 (minterm)

$$M_a(u) = \bigwedge_{r=0}^{k-1} \begin{cases} u_r, & a_r = 1, \\ \neg u_r, & a_r = 0, \end{cases} \quad a \in \{0, 1\}^k,$$

代价为  $O(k2^k)$ . 任意函数  $h: \{0, 1\}^k \rightarrow \{0, 1\}$  都可以写成

$$h(u) = \bigvee_{\substack{a \in \{0, 1\}^k \\ h(a)=1}} M_a(u),$$

因此, 在最小项已经构造好的前提下, 每个  $h$  只需额外  $O(2^k)$  个门.  $k$  元布尔函数总共只有  $2^{2^k}$  种, 所以全部不同的  $h_v$  可以用

$$O(k2^k + 2^{2^k} 2^k)$$

个门计算. 重复出现的  $h_v$  直接复用同一根输出线. 由此得到真值表 (truth table) 的一整行

$$G(u) = (h_v(u))_{v \in \{0, 1\}^{n-k}} \in \{0, 1\}^{2^{n-k}}.$$

最后用  $v$  查表:

$$f(u, v) = \text{LOOKUP}_{n-k}(G(u), v).$$

由 **例 2.3**, 这一步需要  $O(2^{n-k})$  个门. 总规模为

$$O(k2^k + 2^{2^k} 2^k + 2^{n-k}). \quad (2.1)$$

对充分大的  $n$ , 令  $d = n - 2 \log_2 n$ , 并选择整数  $k$  使得

$$\frac{d}{2} < 2^k \leq d.$$

于是

$$2^{2^k} 2^k \leq \frac{2^n}{n^2} n = \frac{2^n}{n}, \quad 2^{n-k} < \frac{2^{n+1}}{n - 2 \log_2 n} = O\left(\frac{2^n}{n}\right).$$

同时  $k2^k = O(n \log n) = O(2^n/n)$ , 故 式 (2.1) 为  $O(2^n/n)$ 。有限多个较小的  $n$  可以吸收到大  $O$  的常数中。最后对  $m$  个输出位分别使用这一构造, 即得

$$|C| = O\left(\frac{m2^n}{n}\right).$$

□

### 2.3.4 计数下界

下面用计数论证 (counting argument) 说明上述关于  $n$  的上界不能继续改进到更低的数量级。

**命题 2.1** (布尔函数的电路下界). 存在常数  $c_0 > 0$ , 使得对所有充分大的  $n$ , 都有某个函数  $f: \{0, 1\}^n \rightarrow \{0, 1\}$  的任意布尔电路至少需要

$$c_0 \frac{2^n}{n}$$

个逻辑门。

证明. 一共有

$$2^{2^n}$$

个函数  $\{0, 1\}^n \rightarrow \{0, 1\}$ 。另一方面, 设一个 NAND-Circ 程序至多有  $s$  行, 并考虑  $s \geq n$  的情形。程序中至多出现  $n + s \leq 2s$  个值。即使把每行都宽松地记为一个三元组, 并把所有不合法的三元组序列也计算在内, 仍存在常数  $c \geq 1$ , 使这类程序的数量至多为

$$2^{cs \log_2 s}.$$

取

$$s = \left\lfloor \frac{2^n}{2cn} \right\rfloor.$$

当  $n$  充分大时,  $s \geq n$  且  $\log_2 s \leq n$ , 所以

$$cs \log_2 s \leq 2^{n-1}, \quad 2^{cs \log_2 s} < 2^{2^n}.$$

程序少于函数, 故至少有一个函数不能由  $s$  行以内的 NAND-Circ 程序计算。再由 NAND 门与 AND、OR、NOT 门之间的常数规模转换, 结论成立。□

因此, 定理 2.2 给出的  $O(2^n/n)$  单输出上界在常数因子意义下是紧的。

## 2.4 程序编码与通用求值

下面约定  $s \geq \max\{n, m, 2\}$ 。一个含  $s$  行、 $n$  个输入和  $m$  个输出的 NAND-Circ 程序至多需要  $3s$  个变量。把它们依次编号为

$$v_0, v_1, \dots, v_{3s-1},$$

其中  $v_0, \dots, v_{n-1}$  存放输入,  $v_n, \dots, v_{n+m-1}$  存放输出。每行程序编码为三元组

$$(a, b, c), \quad v_a \leftarrow \text{NAND}(v_b, v_c).$$

令  $\ell = \lceil \log_2(3s) \rceil$ 。每个下标用  $\ell$  位表示, 故整个程序可以编码为长度  $3s\ell$  的二进制串。

**例 2.4** (通用求值函数). 定义通用求值函数 (universal evaluation function)

$$\text{EVAL}_{s,n,m} : \{0, 1\}^{3s\ell+n} \longrightarrow \{0, 1\}^m.$$

输入写成  $(p, x)$ , 其中  $p$  是一个程序编码,  $x \in \{0, 1\}^n$ 。规定

$$\text{EVAL}_{s,n,m}(p, x) = \begin{cases} P(x), & p \text{ 编码了合法程序 } P, \\ 0^m, & \text{否则.} \end{cases}$$

**定理 2.3** (通用 NAND-Circ 程序). 对任意  $s \geq \max\{n, m, 2\}$ , 都存在计算  $\text{EVAL}_{s,n,m}$  的 NAND-Circ 程序  $U_{s,n,m}$ , 并且

$$|U_{s,n,m}| = O(s^2 \log s).$$

证明. 以下出现的 LOOKUP、MUX 与等值测试都按前文展开为 NAND 门。用  $V \in \{0, 1\}^{3s}$  保存被模拟程序的全部变量值。首先令

$$V^{(0)} = (x_0, \dots, x_{n-1}, 0, \dots, 0).$$

对一个用  $\ell$  位表示的下标  $i$ , 将  $V$  补零到长度  $2^\ell$ , 再调用 例 2.3, 即可实现

$$\text{GET}(V, i) = V_i$$

, 代价为  $O(2^\ell) = O(s)$ 。更新操作逐坐标定义为

$$\text{UPDATE}(V, i, z)_j = \text{MUX}(\text{EQ}_\ell(i, j), V_j, z), \quad 0 \leq j < 3s,$$

其中  $\text{EQ}_\ell$  是  $\ell$  位等值测试 (equality test), 可用  $O(\ell) = O(\log s)$  个门实现。因此

$$|\text{UPDATE}| = O(s \log s).$$

若程序的第  $t$  行编码为  $(a_t, b_t, c_t)$ , 则依次计算

$$\begin{aligned} r_t &= \text{GET}(V^{(t-1)}, b_t), \\ q_t &= \text{GET}(V^{(t-1)}, c_t), \\ z_t &= \text{NAND}(r_t, q_t), \\ V^{(t)} &= \text{UPDATE}(V^{(t-1)}, a_t, z_t). \end{aligned}$$

每轮使用  $O(s \log s)$  个门, 模拟  $s$  行共需  $O(s^2 \log s)$  个门。同时维护“变量是否已经定义”的比特, 并检查下标范围、读取顺序和输出是否已定义, 只会使轮多出常数次 GET、UPDATE 与等值测试。最后用合法性比特屏蔽输出: 合法时输出  $V_n^{(s)}, \dots, V_{n+m-1}^{(s)}$ , 否则输出  $0^m$ 。总规模仍为  $O(s^2 \log s)$ 。□

思考题. 能否把定理 2.3 的规模改进到  $O(s \log s)$ ?

## 第 3 章 布尔函数与语言

### 3.1 从一般函数到布尔函数

一般的计算问题是函数

$$f : \{0, 1\}^* \longrightarrow \{0, 1\}^*.$$

事实上，只研究取值于  $\{0, 1\}$  的布尔函数并不会损失一般性。给定  $f$ ，定义它的逐位查询函数

$$b_f : \{0, 1\}^* \times \mathbb{N} \times \{0, 1\} \longrightarrow \{0, 1\}$$

如下：

$$b_f(x, i, c) = \begin{cases} f(x)_i, & i < |f(x)| \text{ 且 } c = 0, \\ 1, & i < |f(x)| \text{ 且 } c = 1, \\ 0, & i \geq |f(x)|. \end{cases} \quad (3.1)$$

这里从 0 开始给输出位编号。自然数  $i$  和三元组  $(x, i, c)$  均使用固定的二进制编码；具体选用哪一种合理编码，不影响下面的可计算性结论。

**引理 3.1** (逐位查询与原函数等价). 函数  $f$  可计算，当且仅当  $b_f$  可计算。

**证明.** 若有计算  $f$  的程序，先算出  $y = f(x)$ ，再按 [式 \(3.1\)](#) 回答查询，即可计算  $b_f$ 。

反过来，注意到

$$b_f(x, i, 1) = 1 \iff i < |f(x)|, \quad f(x)_i = b_f(x, i, 0) \quad (i < |f(x)|).$$

从  $i = 0$  开始依次查询  $b_f(x, i, 1)$ 。只要结果为 1，就把  $b_f(x, i, 0)$  追加到输出；第一次得到 0 时停止。所得字符串恰为  $f(x)$ 。□

因此，讨论一般字符串函数的可计算性，可以归约为讨论布尔函数的可计算性。



## 3.2 语言与布尔函数

**定义 3.1** (语言与示性函数). 一个语言 (language) 是二进制串集合  $A \subseteq \{0, 1\}^*$ 。对布尔函数  $f: \{0, 1\}^* \rightarrow \{0, 1\}$ , 定义

$$A_f = \{x \in \{0, 1\}^* : f(x) = 1\}.$$

反过来, 语言  $A$  的示性函数 (characteristic function) 定义为

$$\chi_A(x) = \begin{cases} 1, & x \in A, \\ 0, & x \notin A. \end{cases}$$

上述两个变换互为逆映射。因此布尔函数与语言一一对应, 并且

$$f(x) = 1 \iff x \in A_f.$$

## 第 4 章 自动机

自动机 (automaton) 用有限状态描述对输入串的逐位计算。本章先研究确定有限自动机, 再引入非确定有限自动机。

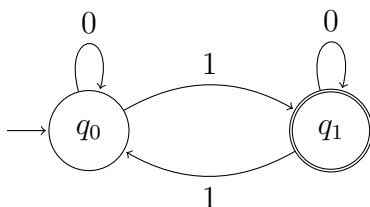
### 4.1 确定有限自动机

**例 4.1** (奇偶校验). 对  $x = x_0x_1 \cdots x_{n-1} \in \{0, 1\}^*$ , 定义

$$\text{PARITY}(x) = x_0 \text{ xor } x_1 \text{ xor } \cdots \text{ xor } x_{n-1}.$$

并约定  $\text{PARITY}(\varepsilon) = 0$ 。

下面的自动机中,  $q_0$  表示目前读到偶数个 1,  $q_1$  表示目前读到奇数个 1。接受状态为  $q_1$ 。



**定义 4.1** (确定有限自动机). 一个确定有限自动机 (deterministic finite automaton, DFA) 是四元组

$$M = (K, s, F, \delta),$$

其中:

1.  $K$  是有限的状态集合 (state set);
2.  $s \in K$  是初始状态 (initial state);
3.  $F \subseteq K$  是接受状态 (accepting state) 的集合;
4.  $\delta : K \times \{0, 1\} \rightarrow K$  是转移函数 (transition function)。

对输入  $x = x_0x_1 \cdots x_{n-1}$ , 自动机产生状态序列

$$s_0 = s, \quad s_{i+1} = \delta(s_i, x_i) \quad (0 \leq i < n).$$

若  $s_n \in F$ , 则称  $M$  接受  $x$ ; 否则称  $M$  拒绝  $x$ 。

**定义 4.2** (计算函数与判定语言). 设  $f: \{0, 1\}^* \rightarrow \{0, 1\}$ ,  $A \subseteq \{0, 1\}^*$ 。

- 如果对每个  $x$ ,  $M$  接受  $x$  当且仅当  $f(x) = 1$ , 则称  $M$  计算  $f$ ;
- 如果对每个  $x$ ,  $M$  接受  $x$  当且仅当  $x \in A$ , 则称  $M$  判定  $A$ 。

**定义 4.3** (DFA 的语言与正则语言). DFA  $M$  接受的全部字符串构成它的语言

$$L(M) = \{x \in \{0, 1\}^* : M \text{ 接受 } x\}.$$

如果存在 DFA  $M$  使得  $A = L(M)$ , 则称语言  $A$  是正则的 (regular)。相应地, 若  $A_f$  正则, 则称布尔函数  $f$  是正则的。

**命题 4.1.** 存在非正则语言。

证明. 每个 DFA 都能编码为有限二进制串, 所以全部 DFA 至多可数。另一方面, 全部语言不可数: 若能把它们枚举为  $A_0, A_1, A_2, \dots$ , 并把  $\{0, 1\}^*$  枚举为  $w_0, w_1, w_2, \dots$ , 则语言

$$D = \{w_i : w_i \notin A_i\}$$

与每个  $A_i$  都不同, 矛盾。因此必有语言不能被任何 DFA 判定。  $\square$

## 4.2 正则语言的封闭性

**定理 4.1** (对并集封闭). 若  $A$  和  $B$  都是正则语言, 则  $A \cup B$  也是正则语言。

证明. 设

$$M_A = (K_A, s_A, F_A, \delta_A), \quad M_B = (K_B, s_B, F_B, \delta_B)$$

分别判定  $A$  和  $B$ 。构造乘积自动机  $M = (K, s, F, \delta)$ , 其中

$$K = K_A \times K_B,$$

$$s = (s_A, s_B),$$

$$F = (F_A \times K_B) \cup (K_A \times F_B),$$

$$\delta((q_A, q_B), a) = (\delta_A(q_A, a), \delta_B(q_B, a)).$$

归纳可知,  $M$  读完任意输入  $x$  后的两个状态分量, 恰好分别是  $M_A$  与  $M_B$  读完  $x$  后的状态。因此

$$M \text{ 接受 } x \iff M_A \text{ 接受 } x \text{ 或 } M_B \text{ 接受 } x \iff x \in A \cup B.$$

□

对语言  $A, B \subseteq \{0, 1\}^*$ , 定义它们的拼接 (concatenation) 为

$$AB = \{xy : x \in A, y \in B\}.$$

**定理 4.2** (对拼接封闭). 若  $A$  和  $B$  都是正则语言, 则  $AB$  也是正则语言。

直接用 DFA 构造拼接并不方便。完整证明将在后文给出; 这里先引入 NFA。

### 4.3 非确定有限自动机

与 DFA 相比, NFA 可以从同一状态沿同一输入转移到多个状态, 也可以进行不消耗输入符号的  $\varepsilon$ -转移。

**定义 4.4** (非确定有限自动机). 一个非确定有限自动机 (nondeterministic finite automaton, NFA) 是四元组

$$N = (K, s, F, \Delta),$$

其中  $K, s, F$  与 DFA 中的含义相同, 而转移关系满足

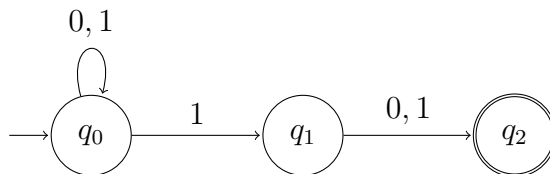
$$\Delta \subseteq K \times (\{0, 1\} \cup \{\varepsilon\}) \times K.$$

NFA 接受字符串  $x$ , 是指存在一条从  $s$  到某个接受状态的路径, 使得路径标签依次串联并删除所有  $\varepsilon$  后恰好得到  $x$ 。

**例 4.2.** 语言

$$A = \{w \in \{0, 1\}^* : |w| \geq 2 \text{ 且 } w \text{ 的倒数第二位为 } 1\}$$

可以由下面的 NFA 判定。在状态  $q_0$  读到 1 时, 自动机既可以留在  $q_0$ , 也可以猜测这个 1 是倒数第二位并转移到  $q_1$ ; 若随后恰好再读一个比特, 便会到达接受状态。



### 4.3.1 NFA 与 DFA 的等价性

**定理 4.3** (NFA 与 DFA 等价). 对每个 NFA, 都存在一个接受相同语言的 DFA; 反之亦然。

证明. DFA 是每一步都只有唯一后继状态、且没有  $\varepsilon$ -转移的特殊 NFA, 故一个 DFA 可以直接看作 NFA。

反过来, 设 NFA 为  $N = (K, s, F, \Delta)$ 。对  $S \subseteq K$ , 记  $E(S)$  为从  $S$  中某个状态出发, 只经过有限次  $\varepsilon$ -转移所能到达的全部状态。构造 DFA

$$D = (\mathcal{P}(K), E(\{s\}), F_D, \delta_D),$$

其中  $\mathcal{P}(K)$  是  $K$  的幂集 (power set), 并令

$$F_D = \{S \subseteq K : S \cap F \neq \emptyset\},$$

$$\delta_D(S, a) = E(\{q' \in K : \text{存在 } q \in S, (q, a, q') \in \Delta\}).$$

这个构造称为状态子集构造 (subset construction)。对输入长度归纳可知,  $D$  读完任意字符串  $x$  后所处的状态, 恰好是  $N$  读完  $x$  后所有可能状态的集合。因此该集合与  $F$  相交, 当且仅当  $N$  有一条接受  $x$  的计算路径。又因为  $K$  有限,  $\mathcal{P}(K)$  也有限, 所以  $D$  确实是 DFA。□

**推论 4.1.** 一个语言是正则语言, 当且仅当它能被某个 NFA 判定。

证明. 由定义 4.3 和定理 4.3 立即得到。□

### 4.3.2 拼接与 Kleene 星

定理 4.2 的证明. 取分别判定  $A$  和  $B$  的 NFA

$$N_A = (K_A, s_A, F_A, \Delta_A), \quad N_B = (K_B, s_B, F_B, \Delta_B),$$

不妨设  $K_A \cap K_B = \emptyset$ 。构造 NFA

$$N = (K_A \cup K_B, s_A, F_B, \Delta),$$

其中

$$\Delta = \Delta_A \cup \Delta_B \cup \{(q, \varepsilon, s_B) : q \in F_A\}.$$

也就是说, 从  $N_A$  的每个接受状态增加一条到  $N_B$  初始状态的  $\varepsilon$ -转移。于是  $N$  接受  $w$ , 当且仅当可以把  $w$  写成  $w = xy$ , 使得  $N_A$  接受  $x$  且  $N_B$  接受  $y$ 。因此  $L(N) = AB$ , 再由推论 4.1 可知  $AB$  正则。□

对语言  $A$ , 递归定义  $A^0 = \{\varepsilon\}$ 、 $A^{n+1} = A^n A$ , 并称

$$A^* = \bigcup_{n \geq 0} A^n$$

为  $A$  的 Kleene 星 (Kleene star)。

**定理 4.4** (对 Kleene 星封闭). 若  $A$  是正则语言, 则  $A^*$  也是正则语言。

证明. 取判定  $A$  的 NFA  $N_A = (K, s, F, \Delta)$ , 并增加一个新状态  $s'$ 。令

$$\begin{aligned} K' &= K \cup \{s'\}, \\ F' &= F \cup \{s'\}, \\ \Delta' &= \Delta \cup \{(s', \varepsilon, s)\} \cup \{(q, \varepsilon, s) : q \in F\}. \end{aligned}$$

则  $N' = (K', s', F', \Delta')$  可以先从  $s'$  进入  $N_A$ , 并在每次到达  $N_A$  的接受状态后重新开始新一轮。新初态  $s'$  本身是接受状态, 所以  $N'$  也接受空串。因此  $L(N') = A^*$ , 由推论 4.1 可知  $A^*$  正则。□

## 4.4 正则表达式

**定义 4.5** (正则表达式). 二进制字母表上的正则表达式 (regular expression) 按下面的规则递归定义:

1.  $\emptyset$ 、0 和 1 是正则表达式, 它们分别描述语言  $\emptyset$ 、 $\{0\}$  和  $\{1\}$ ;
2. 如果  $R_1, R_2$  是正则表达式, 那么  $R_1 \cup R_2$  和  $R_1 R_2$  也是正则表达式, 并且

$$L(R_1 \cup R_2) = L(R_1) \cup L(R_2), \quad L(R_1 R_2) = L(R_1) L(R_2);$$

3. 如果  $R$  是正则表达式, 那么  $R^*$  也是正则表达式, 并且

$$L(R^*) = L(R)^*.$$

在省略括号时, Kleene 星的优先级最高, 拼接次之, 并集最低。另外用  $\varepsilon$  作为  $\emptyset^*$  的简写。

**例 4.3.** 至少含两个 0 的二进制串构成的语言可以写成

$$(0 \cup 1)^* 0 (0 \cup 1)^* 0 (0 \cup 1)^*.$$

**定理 4.5** (正则表达式刻画正则语言). 一个语言是正则语言, 当且仅当它能由正则表达式描述。

证明. 先设语言由正则表达式  $R$  描述. 对  $R$  的递归结构归纳: 三个基本表达式描述的语言都是正则的; 归纳步骤则分别由正则语言对并集、拼接和 Kleene 星的封闭性得到. 因此  $L(R)$  正则。

反过来, 设正则语言  $A$  由某个 NFA  $N$  判定. 先给  $N$  增加一个没有入边的新初态和一个没有出边的唯一新接受态: 从新初态向原初态添加  $\varepsilon$ -转移, 再从每个原接受态向新接受态添加  $\varepsilon$ -转移。

把任意两个状态之间的所有边合并为一条边, 并用这些边标签的并集作为新标签; 没有边时, 标签记为  $\varnothing$ 。随后逐个删除初态与接受态之外的状态. 删除状态  $q$  时, 对每对保留的状态  $p, r$ , 将边  $p \rightarrow r$  的标签更新为

$$R_{pr} \cup R_{pq}(R_{qq})^*R_{qr}.$$

该式分别涵盖了不经过  $q$  的路径, 以及进入  $q$ 、在  $q$  处循环有限次后离开的路径. 归纳可知, 每次删除后, 边标签仍描述原自动机中相应的全部路径. 最后只剩新初态和新接受态, 两者之间的边标签就是一个描述  $A$  的正则表达式。□

## 4.5 正则语言的泵引理

**定理 4.6** (正则语言的泵引理). 若  $L$  是正则语言, 则存在整数  $p \geq 1$ , 使得每个满足  $w \in L$  且  $|w| \geq p$  的字符串都可以写成  $w = xyz$ , 并满足:

1. 对每个整数  $i \geq 0$ , 都有  $xy^iz \in L$ ;
2.  $|y| > 0$ ;
3.  $|xy| \leq p$ 。

这样的  $p$  称为语言  $L$  的一个泵长度 (pumping length)。

证明. 设 DFA  $M = (K, s, F, \delta)$  判定  $L$ , 取  $p = |K|$ 。令  $w = w_0w_1 \cdots w_{n-1} \in L$ , 其中  $n \geq p$ ; 再令  $q_j$  为  $M$  读完  $w$  的前  $j$  位后所处的状态。

序列  $q_0, q_1, \dots, q_p$  含有  $p+1$  个状态. 由抽屉原理 (pigeonhole principle), 存在  $0 \leq r < t \leq p$  使得  $q_r = q_t$ 。取

$$x = w_0 \cdots w_{r-1}, \quad y = w_r \cdots w_{t-1}, \quad z = w_t \cdots w_{n-1}.$$

这里空的下标区间表示空串。

因为  $r < t \leq p$ , 所以  $|y| = t - r > 0$  且  $|xy| = t \leq p$ 。又因为  $q_r = q_t$ , 从  $q_r$  读入  $y$  后仍回到  $q_r$ 。因此无论重复  $y$  多少次, 读入  $z$  之前自动机都处于  $q_r$ ; 于是对每个  $i \geq 0$ ,  $M$  都接受  $xy^iz$ 。□

**例 4.4.** 对正则语言

$$L = 01^*0,$$

$p = 3$  是一个泵长度: 若  $w = 01^n0$  且  $n \geq 1$ , 取  $x = 0$ 、 $y = 1$ 、 $z = 1^{n-1}0$  即可。另一方面,  $p = 2$  不是泵长度, 因为任取满足  $00 = xyz$ 、 $|y| > 0$  和  $|xy| \leq 2$  的分解, 令  $i = 0$  都会得到长度小于 2 的串, 因而不属于  $L$ 。

**例 4.5** ( $\{0^n1^n : n \geq 0\}$  不是正则语言). 令

$$A = \{0^n1^n : n \geq 0\}.$$

假设  $A$  正则, 并令  $p$  为定理 4.6 给出的泵长度。取  $w = 0^p1^p \in A$ 。任取满足  $w = xyz$ 、 $|y| > 0$  且  $|xy| \leq p$  的分解; 由于  $xy$  完全位于前  $p$  个 0 中, 必有  $y = 0^k$ , 其中  $1 \leq k \leq p$ 。令  $i = 0$ , 则

$$xy^0z = 0^{p-k}1^p \notin A,$$

与泵引理矛盾。因此  $A$  不是正则语言。



## 第 5 章 PDA 与语法

有限自动机只有有限个状态，无法记录任意长的中间信息。本章在 NFA 上增加一个栈，得到下推自动机，并说明它与上下文无关文法具有相同的表达能力。

### 5.1 下推自动机

**定义 5.1** (下推自动机). 一个下推自动机 (pushdown automaton, PDA) 是四元组

$$P = (K, s, F, \Delta),$$

其中  $K$  是有限状态集， $s \in K$  是初始状态， $F \subseteq K$  是接受状态集，而  $\Delta$  是有限的转移关系：

$$\Delta \subseteq (K \times (\{0, 1\} \cup \{\varepsilon\}) \times \{0, 1\}^*) \times (K \times \{0, 1\}^*).$$

把  $\Delta$  中的元素写成

$$(p, a, \alpha) \longrightarrow (q, \beta).$$

它表示：自动机在状态  $p$  读取  $a$ ，从栈顶弹出  $\alpha$ ，压入  $\beta$ ，然后进入状态  $q$ 。这里  $a \in \{0, 1\} \cup \{\varepsilon\}$ ，而  $\alpha, \beta \in \{0, 1\}^*$ ； $a = \varepsilon$  表示不读取输入。约定栈顶位于栈串的左端。

**定义 5.2** (格局与一步转移). PDA  $P$  的一个格局 (configuration) 是三元组

$$(p, x, \gamma) \in K \times \{0, 1\}^* \times \{0, 1\}^*,$$

分别记录当前状态、尚未读取的输入和栈内容。若  $(p, a, \alpha) \rightarrow (q, \beta)$  是一条转移，则

$$(p, ax, \alpha\gamma) \vdash_P (q, x, \beta\gamma),$$

其中约定  $\varepsilon x = x$ 。记  $\vdash_P^*$  为  $\vdash_P$  的自反传递闭包。

**定义 5.3** (接受与上下文无关语言). PDA  $P = (K, s, F, \Delta)$  接受字符串  $w$ , 是指存在  $q \in F$  使得

$$(s, w, \varepsilon) \vdash_P^* (q, \varepsilon, \varepsilon).$$

PDA 接受的语言记为

$$L(P) = \{w \in \{0, 1\}^* : P \text{ 接受 } w\}.$$

如果  $L = L(P)$  对某个 PDA  $P$  成立, 则称  $L$  是上下文无关语言 (context-free language, CFL)。

**例 5.1** (含有相同数量的 0 和 1). 令

$$L_ = \{w \in \{0, 1\}^* : \#_0(w) = \#_1(w)\}.$$

取只有一个状态  $q$  的 PDA, 并令  $q$  同时为初始状态和接受状态。它具有以下四条转移:

$$\begin{aligned} (q, 0, \varepsilon) &\longrightarrow (q, 0), & (q, 0, 1) &\longrightarrow (q, \varepsilon), \\ (q, 1, \varepsilon) &\longrightarrow (q, 1), & (q, 1, 0) &\longrightarrow (q, \varepsilon). \end{aligned}$$

读到一个比特时, 自动机可以把它压栈; 如果栈顶是相反的比特, 也可以将二者抵消。若一条计算路径以空栈结束, 每次压入的 0 都与一个读入的 1 配对, 反之亦然, 所以输入中两种比特数量相同。

反过来, 每次优先抵消相反的栈顶, 否则把当前比特压栈。此时栈中始终只含同一种比特, 栈高等于已读前缀中两种比特数量之差的绝对值。因此输入属于  $L_ =$  时, 最终栈为空。故这个 PDA 恰好接受  $L_ =$ 。

对字符串  $w = w_0w_1 \cdots w_{n-1}$ , 记

$$w^R = w_{n-1} \cdots w_1w_0$$

为它的反转 (reverse)。

**例 5.2** (偶数长度的回文串). 语言

$$L_{\text{even-pal}} = \{ww^R : w \in \{0, 1\}^*\}$$

可由状态集  $K = \{\ell, r\}$ 、初态  $\ell$ 、接受状态集  $\{r\}$  的 PDA 接受。它的转移为

$$\begin{aligned}
(\ell, 0, \varepsilon) &\longrightarrow (\ell, 0), & (\ell, 1, \varepsilon) &\longrightarrow (\ell, 1), \\
(\ell, \varepsilon, \varepsilon) &\longrightarrow (r, \varepsilon), \\
(r, 0, 0) &\longrightarrow (r, \varepsilon), & (r, 1, 1) &\longrightarrow (r, \varepsilon).
\end{aligned}$$

自动机在状态  $\ell$  把前半段压栈，并非确定地猜测中点；进入  $r$  后，后半段必须逐位匹配并弹出栈顶。因此它恰好接受形如  $ww^R$  的字符串。

## 5.2 上下文无关文法

**定义 5.4** (上下文无关文法). 二进制字母表上的一个上下文无关文法 (context-free grammar, CFG) 是三元组

$$G = (V, S, R),$$

其中：

1.  $V$  是包含终结符  $0, 1$  的有限符号集；
2.  $S \in V \setminus \{0, 1\}$  是开始符号 (start symbol)；
3.  $R \subseteq (V \setminus \{0, 1\}) \times V^*$  是有限的产生式 (production rule) 集合。

产生式  $(A, u) \in R$  通常写作  $A \rightarrow u$ 。 $V \setminus \{0, 1\}$  中的符号称为非终结符 (nonterminal)。

**定义 5.5** (推导与生成语言). 设  $x, y, u \in V^*$ ,  $A \in V \setminus \{0, 1\}$ 。如果  $A \rightarrow u$  是  $G$  的产生式，则记

$$xAy \Rightarrow_G xuy.$$

记  $\Rightarrow_G^*$  为  $\Rightarrow_G$  的自反传递闭包。若  $S \Rightarrow_G^* w$ ，则称  $G$  生成  $w$ ； $G$  生成的语言为

$$L(G) = \{w \in \{0, 1\}^* : S \Rightarrow_G^* w\}.$$

**例 5.3** (回文串文法). 取  $V = \{0, 1, S\}$ ，产生式为

$$S \rightarrow \varepsilon \mid 0 \mid 1 \mid 0S0 \mid 1S1.$$

每次递归产生式都在两端添加相同的比特，而三个基本产生式生成长度为 0 或 1 的回文串。因此

$$L(G) = \{w \in \{0, 1\}^* : w = w^R\}.$$

### 5.3 PDA 与 CFG 的等价性

为了给出等价性的构造，先把 PDA 的转移化为统一的形式。

**定义 5.6** (简单 PDA). 如果一个 PDA 只有一个接受状态，并且每条转移

$$(p, a, \alpha) \longrightarrow (q, \beta)$$

都恰好执行以下一种操作，则称它是简单 PDA (simple PDA)：

1.  $\alpha = \varepsilon$  且  $|\beta| = 1$ ，即压入一个比特；
2.  $|\alpha| = 1$  且  $\beta = \varepsilon$ ，即弹出一个比特。

**引理 5.1** (PDA 的简化). 每个 PDA 都存在一个接受相同语言的简单 PDA。

**证明.** 对每条广义转移引入只供它使用的中间状态。先逐位弹出  $\alpha$ ，再按逆序逐位压入  $\beta$ ；输入符号只在这一串新转移的第一步读取。若  $\alpha = \beta = \varepsilon$ ，则用“压入一个 0，再立即弹出”代替原转移。这样每一步都只压入或弹出一个比特。

最后增加唯一的新接受状态。把原接受状态改为普通状态，并从每个原接受状态经由“压入一个 0，再弹出”连接到新接受状态。由于接受时输入和栈都必须为空，这些修改既不会增加也不会删去任何被接受的字符串。  $\square$

**定理 5.1** (PDA 与 CFG 等价). 对语言  $L \subseteq \{0,1\}^*$ ，以下两件事等价：

1. 存在 PDA  $P$  使得  $L = L(P)$ ；
2. 存在 CFG  $G$  使得  $L = L(G)$ 。

**证明.** 先设  $G = (V, S, R)$  是生成  $L$  的 CFG。因为  $V$  有限，可以取某个定长单射编码  $c: V \rightarrow \{0,1\}^m$ ，并把  $c$  按串联扩展到  $V^*$ ，其中  $c(\varepsilon) = \varepsilon$ 。构造 PDA  $P_G$ ，其状态集为  $\{s_0, q\}$ ，初态为  $s_0$ ，唯一接受状态为  $q$ 。加入转移

$$\begin{aligned} (s_0, \varepsilon, \varepsilon) &\longrightarrow (q, c(S)), \\ (q, \varepsilon, c(A)) &\longrightarrow (q, c(u)) & (A \rightarrow u \in R), \\ (q, b, c(b)) &\longrightarrow (q, \varepsilon) & (b \in \{0,1\}). \end{aligned}$$

第一条转移把开始符号压栈，第二类转移模拟最左推导，第三类转移将已生成的终结符与输入逐位匹配。因此  $P_G$  接受  $w$  当且仅当  $S \Rightarrow_G^* w$ ，故它接受  $L$ 。

反过来，由引理 5.1，只需考虑简单 PDA  $P = (K, s, \{f\}, \Delta)$ 。对每对状态  $p, q \in K$  引入非终结符  $A_{pq}$ ，并取开始符号  $A_{sf}$ 。加入以下产生式：

$$\begin{aligned} A_{pp} &\rightarrow \varepsilon & (p \in K), \\ A_{pq} &\rightarrow A_{pr}A_{rq} & (p, q, r \in K). \end{aligned}$$

此外, 若  $a, b \in \{0, 1\} \cup \{\varepsilon\}$ 、 $d \in \{0, 1\}$ , 且  $P$  含有一条压栈转移和一条与之匹配的弹栈转移

$$(p, a, \varepsilon) \longrightarrow (p', d), \quad (q', b, d) \longrightarrow (q, \varepsilon),$$

则加入

$$A_{pq} \rightarrow aA_{p'q'}b,$$

其中  $a = \varepsilon$  或  $b = \varepsilon$  时省略相应符号。

对推导树和计算步数归纳可得

$$A_{pq} \Rightarrow_G^* w \iff (p, w, \varepsilon) \vdash_P^* (q, \varepsilon, \varepsilon).$$

第一类产生式对应空计算, 第二类对应在栈为空处把计算分成两段, 第三类对应一次压栈及其匹配的弹栈。取  $p = s, q = f$ , 便有  $L(G) = L(P)$ 。□

## 第 6 章 图灵机

PDA 仍有其无法识别的语言，例如

$$\{0^n 1^n 0^n : n \in \mathbb{N}\}.$$

为了描述更一般的计算，本章引入图灵机。它具有一条无限纸带，可以在有限控制下读写纸带并向左右移动。

### 6.1 图灵机模型

**定义 6.1** (图灵机). 一台图灵机 (Turing machine, TM) 是四元组

$$M = (K, \Gamma, s, \delta),$$

其中  $K$  是有限状态集,  $s \in K$  是初始状态,  $\Gamma$  是有限的纸带字母表 (tape alphabet), 且

$$\{0, 1, \triangleright, \sqcup\} \subseteq \Gamma.$$

这里  $\triangleright$  是纸带左端标记,  $\sqcup$  是空白符。转移函数为

$$\delta : K \times \Gamma \longrightarrow K \times \Gamma \times \{L, R, S, H\},$$

其中  $L, R, S, H$  分别表示左移、右移、原地不动和停机 (halt)。为保持左端标记不变，约定读到  $\triangleright$  时不能左移或改写它，且其他位置不能写入  $\triangleright$ 。

设输入为  $x = x_0 x_1 \cdots x_{n-1} \in \{0, 1\}^*$ 。纸带是函数  $T : \mathbb{N} \rightarrow \Gamma$ ，其初始内容为

$$T(0) = \triangleright, \quad T(j+1) = x_j \quad (0 \leq j < n), \quad T(j) = \sqcup \quad (j > n).$$

机器从状态  $s$ 、位置  $i = 0$  开始。若当前状态为  $q$ ，且

$$\delta(q, T(i)) = (q', a, D),$$

则先令  $T(i) \leftarrow a$ 、 $q \leftarrow q'$ ，再按照

$$i \leftarrow \begin{cases} i - 1, & D = L, \\ i + 1, & D = R, \\ i, & D = S \end{cases}$$

移动读写头；若  $D = H$ ，则写入后立即停机。

机器停机时，从左端标记右侧开始读取，到第一个空白符为止。若读到的符号全是比特，就把这个比特串记为  $M(x)$ ；若机器不停机，或停机时输出区域含有其他工作符号，则记  $M(x) = \perp$ 。

**定义 6.2** (可计算函数). 若对每个  $x \in \{0, 1\}^*$  都有  $M(x) = f(x)$ ，则称图灵机  $M$  计算函数  $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ ，并称  $f$  是可计算函数 (computable function)。特别地，计算一个函数的图灵机必须在每个输入上停机并产生合法输出。

## 6.2 NAND-TM 编程语言

图灵机适合刻画计算能力，却不适合直接编写较长的程序。下面引入与它等价、但更接近编程语言的 NAND-TM 模型。

**定义 6.3** (NAND-TM 程序). 一个 NAND-TM 程序包含有限个布尔标量变量、有限个以非负整数  $i \in \mathbb{N}$  为下标的布尔数组，以及一个有限的指令块。与图灵机相同，程序不允许从  $i = 0$  继续左移。

指令块中的普通指令形如

$$u \leftarrow \text{NAND}(v, w),$$

其中变量可以是标量，也可以是当前下标处的数组元素  $A[i]$ 。每轮执行完这些指令后，程序执行

$$\text{MODANDJUMP}(a, b),$$

并按下表修改  $i$ ，然后回到指令块开头：

| $(a, b)$ | 操作                   |
|----------|----------------------|
| $(1, 1)$ | $i \leftarrow i + 1$ |
| $(0, 1)$ | $i \leftarrow i - 1$ |
| $(1, 0)$ | $i \leftarrow i$     |
| $(0, 0)$ | 停机                   |

输入和输出分别存放在指定数组  $X$  与  $Y$  中；为区分字符串末尾，采用一个固定的自定界编码 (self-delimiting encoding)。除输入数组中由编码指定的位置外，所有变量和数组元素的初值均为 0。

自定界编码的具体选择不影响模型能计算哪些函数；它只负责区分例如 1 与 10 这类具有不同长度的输入。

**定理 6.1** (图灵机与 NAND-TM 等价). 图灵机与 *NAND-TM* 能计算完全相同的函数。

证明. 先用 NAND-TM 模拟图灵机  $M = (K, \Gamma, s, \delta)$ 。用

$$k = \lceil \log_2 |K| \rceil, \quad \ell = \lceil \log_2 |\Gamma| \rceil$$

个比特分别编码当前状态和一个纸带符号。状态编码存入  $k$  个标量；纸带每个位置的符号编码，按位存入  $\ell$  个布尔数组。下标  $i$  就是读写头的位置。再用两个比特编码移动方向：

$$L \leftrightarrow 01, \quad R \leftrightarrow 11, \quad S \leftrightarrow 10, \quad H \leftrightarrow 00.$$

转移函数  $\delta$  在有限集合上，因此其编码是一个有限布尔函数，可以由 NAND 电路实现。每轮指令先计算新状态和新纸带符号，再把方向编码交给 MODANDJUMP；故一轮恰好模拟图灵机的一步。

反过来，考虑一个有  $k$  个标量、 $\ell$  个数组的 NAND-TM 程序。图灵机在有限状态中记录  $k$  个标量的取值以及当前执行到的指令；把同一下标处的  $\ell$  个数组元素合并成纸带字母  $(A_1[i], \dots, A_\ell[i])$ 。每条 NAND 指令和 MODANDJUMP 的四种情形都是有限转移，因而均可由图灵机的若干步完成。这样图灵机便可逐轮模拟该程序。  $\square$

### 6.2.1 NAND-TM 程序中的语法糖

为了便于书写，可以加入不会改变计算能力的语法糖 (syntactic sugar)。

- **GOTO 与循环**：用若干标量编码当前行号，并用 NAND 指令更新行号，就可以实现跳转；循环则由跳回较早的行得到。
- **多维数组**：通过一个可计算的配对函数  $\langle \cdot, \cdot \rangle : \mathbb{N}^2 \rightarrow \mathbb{N}$ ，可将  $A[i, j]$  存为一维数组元素  $A'[\langle i, j \rangle]$ 。更高维数组同理。

这些写法可能改变运行时间，但不会改变可计算函数的范围。

## 6.3 NAND-RAM

实际计算机通常允许直接访问给定地址，而不必让读写头逐格移动。随机存取机 (random-access machine, RAM) 正是对此的抽象。

**定义 6.4** (NAND-RAM 程序). *NAND-RAM* 是在 NAND-TM 基础上加入下列操作得到的编程模型：



1. 用有限二进制串表示的整数变量与寄存器；
2. 按地址随机读取和写入数组元素；
3. 基本的布尔、算术与流程控制指令。

每条指令只操作有限长的数据，并由有限算法给出其语义。

**定理 6.2** (NAND-TM 与 NAND-RAM 等价). *NAND-TM* 与 *NAND-RAM* 能计算完全相同的函数。

证明思路. *NAND-RAM* 可以直接保存 *NAND-TM* 的标量、数组和下标，并逐条模拟其指令。反方向上，图灵机可在纸带上编码 *NAND-RAM* 的寄存器与已使用的内存单元；每次随机访问时扫描这份编码，找到相应地址后读取或改写。基本算术和流程控制都能分解为有限的逐位操作。因此 *NAND-RAM* 可由图灵机模拟，再结合 [定理 6.1](#) 即得结论。  $\square$

## 6.4 Church–Turing 论题

*Church–Turing* 论题 (*Church–Turing thesis*) 断言：凡是能够由有效、机械的步骤完成的计算，都可以由图灵机完成。换言之，图灵机刻画了“算法可计算”的本质。

这里使用“论题”而不是“定理”，因为“有效机械步骤”是一个先于形式模型的直观概念，无法在不先选定计算模型的前提下成为可证明的数学命题。大量彼此独立提出的计算模型都与图灵机等价，这是支持该论题的主要证据。

## 第 7 章 可计算性

上一章定义了图灵机。本章研究可计算性 (computability): 首先说明一台图灵机可以模拟任意其他图灵机, 然后研究哪些函数无法由任何图灵机计算。

### 7.1 通用图灵机

为了把机器本身作为输入, 需要先约定图灵机的编码。对  $M = (K, \Gamma, s, \delta)$ , 分别把  $K$  与  $\Gamma$  中的元素编号, 再把每条转移

$$\delta(q, a) = (q', b, D)$$

写成五元组  $(q, a, q', b, D)$ 。利用自定界编码依次记录状态数、字母表大小、初始状态和全部转移, 就得到  $M$  的编码  $\langle M \rangle \in \{0, 1\}^*$ 。这种编码具有以下性质: 是否合法可以判定, 并且可以从合法编码中恢复  $M$  的全部数据。

下文用  $\langle u, v \rangle$  表示两个字符串的固定自定界编码, 并把  $U(\langle m, x \rangle)$  简写为  $U(m, x)$ 。

**定理 7.1** (通用图灵机). 存在一台图灵机  $U$ , 使得对任意  $m, x \in \{0, 1\}^*$ ,

$$U(m, x) = \begin{cases} M(x), & m = \langle M \rangle \text{ 是某台图灵机的合法编码,} \\ 0, & m \text{ 不是合法编码.} \end{cases}$$

第一种情形中, 若  $M$  在  $x$  上不停机, 则  $U$  也不停机。

证明.  $U$  先检查并解析  $m$ 。若编码不合法, 就输出 0; 否则从中恢复  $M = (K, \Gamma, s, \delta)$ 。随后  $U$  在自己的纸带上记录  $M$  的当前状态、读写头位置和纸带内容, 并根据编码中的转移表逐步更新这三个部分。每一轮模拟  $M$  的一步, 所以两台机器同时停机且输出相同。□

这样的  $U$  称为通用图灵机 (universal Turing machine)。它表明我们不需要为每个算法制造一台新的物理机器: 只需把程序编码后交给同一台机器执行。

## 7.2 不可计算函数

**定理 7.2** (不可计算函数的存在性). 存在函数  $F: \{0,1\}^* \rightarrow \{0,1\}$  不是可计算函数。

证明一：计数。每台图灵机都有一个有限二进制编码，所以图灵机的集合至多可数，因而它们至多能计算可数多个函数。另一方面， $\{0,1\}^*$  是可数无限集，因此从  $\{0,1\}^*$  到  $\{0,1\}$  的函数共有

$$2^{\aleph_0}$$

个，是不可数的。故其中必有函数不能由图灵机计算。  $\square$

下面的对角化 (diagonalization) 证明给出一个具体的不可计算函数。若  $m$  是合法图灵机编码，记其对应的机器为  $M_m$ 。定义

$$F^*(m) = \begin{cases} 0, & m \text{ 合法且 } M_m(m) = 1, \\ 1, & \text{其他情形.} \end{cases} \quad (7.1)$$

“其他情形”包括  $m$  非法、 $M_m(m)$  不停机，以及它停机但输出不是 1。

证明二：对角化。假设某台图灵机  $M_e$  计算  $F^*$ ，其中  $e = \langle M_e \rangle$ 。把  $e$  输入它自身。若  $M_e(e) = 1$ ，则由式 (7.1) 得  $F^*(e) = 0$ ；若  $M_e(e) = 0$ ，则同一式给出  $F^*(e) = 1$ 。两种情形都与  $M_e(e) = F^*(e)$  矛盾，故  $F^*$  不可计算。  $\square$

## 7.3 停机问题

**定义 7.1** (停机问题). 停机问题 (halting problem) 是函数

$$\text{HALT}(m, x) = \begin{cases} 1, & m = \langle M \rangle \text{ 且 } M \text{ 在输入 } x \text{ 上停机,} \\ 0, & \text{其他情形.} \end{cases}$$

**定理 7.3** (停机问题不可计算). 函数 HALT 不可计算。

证明。假设图灵机  $H$  计算 HALT。由它构造一台机器  $D$ ：输入  $m$  后，先计算  $H(m, m)$ 。若结果为 0，则输出 1；若结果为 1，则按照定理 7.1 的方法逐步模拟  $M_m(m)$  直至其停机。若模拟得到的输出恰为 1，就输出 0；否则输出 1。

当  $H(m, m) = 1$  时，被模拟的机器必定停机，所以  $D$  在每个输入上都停机。对照式 (7.1) 可知  $D$  恰好计算  $F^*$ ，这与定理 7.2 矛盾。  $\square$

只询问机器在某个固定输入上是否停机，也不会使问题变得可计算。定义

$$\text{HALT}_0(m) = \begin{cases} 1, & m = \langle M \rangle \text{ 且 } M \text{ 在输入 } 0 \text{ 上停机,} \\ 0, & \text{其他情形.} \end{cases}$$

**定理 7.4** (固定输入上的停机问题). 函数  $\text{HALT}_0$  不可计算。

证明. 给定  $m, x$ , 可以有效构造图灵机  $N_{m,x}$ : 它忽略自己的输入, 若  $m$  合法就模拟  $M_m(x)$ , 否则永远循环。于是

$$\text{HALT}(m, x) = \text{HALT}_0(\langle N_{m,x} \rangle).$$

若  $\text{HALT}_0$  可计算, 等式右侧便给出了停机问题的算法, 与 [定理 7.3](#) 矛盾。  $\square$

不可计算性不仅涉及机器是否停机, 也涉及机器计算了什么函数。

**例 7.1** (识别常零函数). 令  $Z(x) = 0$ , 并定义

$$\text{ZERO}(m) = \begin{cases} 1, & m = \langle M \rangle \text{ 且 } M \text{ 计算函数 } Z, \\ 0, & \text{其他情形.} \end{cases}$$

函数  $\text{ZERO}$  不可计算。

证明. 假设  $\text{ZERO}$  可计算. 给定  $m, x$ , 构造机器  $N_{m,x}$ : 在任意输入  $y$  上, 它先模拟  $M_m(x)$ ; 若模拟停机, 就输出 0. 若  $m$  非法, 则令  $N_{m,x}$  永远循环。因此

$$\text{HALT}(m, x) = \text{ZERO}(\langle N_{m,x} \rangle),$$

这将使停机问题可计算, 产生矛盾。  $\square$

## 7.4 归约

**定义 7.2** (归约). 设  $F, G: \{0, 1\}^* \rightarrow \{0, 1\}$ 。从  $F$  到  $G$  的一个归约 (reduction) 是一个可计算函数  $R: \{0, 1\}^* \rightarrow \{0, 1\}^*$ , 满足

$$F(x) = G(R(x)) \quad (x \in \{0, 1\}^*).$$

若存在这样的  $R$ , 记为  $F \leq_m G$ , 并称  $F$  可多对一归约到  $G$  (many-one reducible)。

**推论 7.1.** 若  $F \leq_m G$  且  $G$  可计算, 则  $F$  可计算。

证明. 先计算  $R(x)$ , 再计算  $G(R(x))$ , 所得结果就是  $F(x)$ 。  $\square$

**推论 7.2.** 若  $F \leq_m G$  且  $F$  不可计算, 则  $G$  不可计算。

证明. 这是 [推论 7.1](#) 的逆否命题。  $\square$

前面两个证明已经隐含使用了归约: 它们分别给出

$$\text{HALT} \leq_m \text{HALT}_0, \quad \text{HALT} \leq_m \text{ZERO}.$$

## 7.5 Rice 定理

机器的语义性质 (semantic property) 只取决于它所计算的函数, 而不取决于状态名称、转移表写法等实现细节。

**定义 7.3** (语义且非平凡的性质). 图灵机的性质  $P$  称为语义的, 如果任意两台计算同一个偏函数 (partial function) 的机器  $M, M'$  都满足

$$P(M) = P(M').$$

若存在图灵机  $M_0, M_1$  使得  $P(M_0) = 0$  且  $P(M_1) = 1$ , 则称  $P$  是非平凡的 (non-trivial)。

对这样的性质, 考虑其判定函数

$$\chi_P(m) = \begin{cases} P(M), & m = \langle M \rangle, \\ 0, & m \text{ 不是合法编码}. \end{cases}$$

**定理 7.5** (Rice 定理). 若  $P$  是图灵机的语义且非平凡的性质, 则  $\chi_P$  不可计算。

证明. 令  $M_\infty$  为在每个输入上都永远循环的机器。必要时把  $P$  换成其补性质  $1 - P$ ; 判定二者是等价的。因此不妨设  $P(M_\infty) = 0$ 。由非平凡性可取一台机器  $M_{\text{yes}}$ , 使得  $P(M_{\text{yes}}) = 1$ 。

给定  $m, x$ , 有效构造机器  $N_{m,x}$ 。它在输入  $y$  上执行:

1. 若  $m$  非法则永远循环; 否则模拟  $M_m(x)$ ;
2. 上一步停机后, 模拟  $M_{\text{yes}}(y)$  并原样输出其结果。

若  $\text{HALT}(m, x) = 0$ , 则  $N_{m,x}$  与  $M_\infty$  计算同一个处处无定义的偏函数, 所以  $P(N_{m,x}) = 0$ 。若  $\text{HALT}(m, x) = 1$ , 则  $N_{m,x}$  与  $M_{\text{yes}}$  计算同一个偏函数, 所以  $P(N_{m,x}) = 1$ 。因此

$$\text{HALT}(m, x) = \chi_P(\langle N_{m,x} \rangle),$$

即  $\text{HALT} \leq_m \chi_P$ 。由 [定理 7.3](#) 以及 [推论 7.2](#) 可知  $\chi_P$  不可计算。 □

## 7.6 自引用与递归定理

程序可以把程序编码作为普通字符串处理。固定上一节的图灵机编码后, 存在以下两个可计算的代码生成函数:

- $q(w)$  返回机器  $A_w$  的编码； $A_w$  在输入  $x$  上输出二元组  $\langle w, x \rangle$ ；
- $c(a, b)$  返回顺次运行编码为  $a, b$  的两台机器所得机器的编码。

它们都是可计算的，因为只需把  $w$  或两张转移表填入一个固定的程序模板。

**Pseudocode: Hardwiring machine  $A_w$  (input  $x$ )**

1. Preserve  $x$  and write the fixed string  $w$  on a work tape.
2. Output  $\langle w, x \rangle$  and halt.

**例 7.2** (自打印图灵机). 存在一台图灵机  $R$ ，它在任意输入上都输出自己的编码  $\langle R \rangle$ 。

证明. 构造机器  $B$ ：它在输入  $\langle w, x \rangle$  上输出  $c(q(w), w)$ 。令

**Pseudocode: Machine  $B$  (input  $\langle w, x \rangle$ )**

1. Compute  $a \leftarrow q(w)$ , the encoding of  $A_w$ .
2. Compute  $r_w \leftarrow c(a, w)$ .
3. Output  $r_w$  and halt.

$$b = \langle B \rangle, \quad r = c(q(b), b),$$

并令  $R$  为编码  $r$  所表示的机器。 $R$  先运行  $A_b$ ，把输入  $x$  变为  $\langle b, x \rangle$ ，再运行  $B$ ，所以

**Pseudocode: Self-printing machine  $R$  (input  $x$ )**

1. Run  $A_b(x)$  to obtain  $\langle b, x \rangle$ .
2. Run  $B$  on the result and return its output.

$$R(x) = c(q(b), b) = r = \langle R \rangle.$$

□

同一个技巧允许程序在运行时取得自己的编码。

**定理 7.6** (递归定理). 设图灵机  $T$  接受两个输入。存在图灵机  $R$ ，使得对每个  $x \in \{0, 1\}^*$ ,

$$R(x) = T(\langle R \rangle, x).$$

等式按偏函数理解：一侧不停机当且仅当另一侧不停机。

证明. 由  $T$  构造机器  $B_T$ 。它在输入  $\langle w, x \rangle$  上先计算

$$r_w = c(q(w), w),$$

再运行  $T(r_w, x)$ 。令  $b = \langle B_T \rangle$ ，并令  $R$  是编码  $r = c(q(b), b)$  所表示的机器。对任意  $x$ ，第一阶段  $A_b$  产生  $\langle b, x \rangle$ ，随后  $B_T$  计算的  $r_b$  正好等于  $r = \langle R \rangle$ 。因此

**Pseudocode: Machine  $B_T$  in the recursion theorem (input  $\langle w, x \rangle$ )**

1. Compute  $a \leftarrow q(w)$ .
2. Compute  $r_w \leftarrow c(a, w)$ .
3. Simulate  $T(r_w, x)$  and return its output.

**Pseudocode: Fixed-point machine  $R$  (input  $x$ )**

1. Run  $A_b(x)$  to obtain  $\langle b, x \rangle$ , where  $b = \langle B_T \rangle$ .
2. Run  $B_T$  on the result and return its output.

$$R(x) = B_T(b, x) = T(\langle R \rangle, x).$$

□

**例 7.3** (用递归定理重证停机问题). 假设存在计算 HALT 的机器  $H$ 。构造机器  $T$ ，使得

$$T(m, x) = \begin{cases} \text{永远循环}, & H(m, x) = 1, \\ 0, & H(m, x) = 0. \end{cases}$$

等价地，由递归定理得到的自引用机器可写成：

**Pseudocode: Contradiction machine  $R$  (input  $x$ )**

1. Obtain its own encoding  $m \leftarrow \langle R \rangle$ .
2. Compute  $h \leftarrow H(m, x)$ .
3. If  $h = 1$ , loop forever; otherwise, output 0 and halt.

由 [定理 7.6](#)，存在  $R$  满足  $R(x) = T(\langle R \rangle, x)$ 。若  $H(\langle R \rangle, x) = 1$ ，则  $R(x)$  不停机；若其值为 0，则  $R(x)$  停机。两种情形都与  $H$  的输出矛盾，故 HALT 不可计算。

### 7.6.1 最小机器与不动点

记  $M \simeq N$  表示两台机器计算同一个偏函数。称机器  $M$  是最小的 (minimal)，如果

$$M' \simeq M \implies |\langle M' \rangle| \geq |\langle M \rangle|.$$

定义

$$\text{MIN}(m) = \begin{cases} 1, & m = \langle M \rangle \text{ 且 } M \text{ 是最小机器,} \\ 0, & \text{其他情形.} \end{cases}$$

**例 7.4** (识别最小机器). 函数 MIN 不可计算。

证明. 假设 MIN 可计算。最小机器的编码长度没有上界：函数  $x \mapsto 0^n$  随  $n$  改变而彼此不同，而长度不超过任意给定常数的编码只有有限多个。

构造机器  $T$ 。在输入  $\langle m, x \rangle$  上，它按长度枚举字符串  $s$ ，直至找到

$$\text{MIN}(s) = 1 \quad \text{且} \quad |s| > |m|,$$

然后在  $x$  上模拟编码为  $s$  的机器。由无界性，这个搜索一定结束。根据 [定理 7.6](#)，存在  $R$  使得  $R(x) = T(\langle R \rangle, x)$ 。设搜索得到  $s$ ，则

**Pseudocode: Contradiction machine  $R$  for recognizing minimal machines**  
(input  $x$ )

1. Obtain its own encoding  $m \leftarrow \langle R \rangle$ .
2. Enumerate  $s \in \{0, 1\}^*$  in order of length until  $\text{MIN}(s) = 1$  and  $|s| > |m|$ .
3. Simulate  $M_s$  on  $x$  and return its output.

$$R \simeq M_s, \quad |\langle R \rangle| < |s|.$$

这与  $M_s$  是最小机器矛盾。 □

递归定理也常写成下面的不动点 (fixed point) 形式。

**推论 7.3** (不动点定理). 设  $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$  是可计算函数，并且把合法图灵机编码映射为合法图灵机编码。则存在图灵机  $F$ ，使得

$$F \simeq M_{f(\langle F \rangle)}.$$

证明. 令  $T(m, x)$  模拟编码为  $f(m)$  的机器在  $x$  上的计算。对  $T$  应用 [定理 7.6](#) 即得所需的  $F$ 。

**Pseudocode: Fixed-point machine  $F$  (input  $x$ )**

1. Obtain its own encoding  $e \leftarrow \langle F \rangle$ .
  2. Compute  $m \leftarrow f(e)$ .
  3. Simulate  $M_m$  on  $x$  and return its output.
- 

## 7.7 有效证明系统与不完备性

**定义 7.4** (有效证明系统). 设  $\mathcal{T} \subseteq \{0, 1\}^*$  是一组真命题的编码。它的一个有效证明系统 (effective proof system) 是图灵机

$$V : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\},$$

其中  $V(x, p) = 1$  表示  $p$  是命题  $x$  的证明，并满足：



1. **有效性 (effectiveness)**: 对每个  $x, p$ ,  $V(x, p)$  都停机;
2. **可靠性 (soundness)**: 若  $x \notin \mathcal{T}$ , 则对每个  $p$  都有  $V(x, p) = 0$ 。

如果对每个  $x \in \mathcal{T}$  都存在  $p$  使  $V(x, p) = 1$ , 则称  $V$  是完备的 (complete)。

**定理 7.7** (抽象不完备性). 存在一组真命题  $\mathcal{T}$ , 它没有可靠且完备的有效证明系统。

证明. 对每台图灵机  $M$ , 考虑两个命题

$$h_M = \text{“}M \text{ 在输入 } 0 \text{ 上停机”}, \quad n_M = \text{“}M \text{ 在输入 } 0 \text{ 上不停机”},$$

并令  $\mathcal{T}$  包含其中所有真命题。假设  $V$  是可靠且完备的有效证明系统。给定字符串  $m$ , 若它不是合法机器编码就输出 0; 否则写成  $m = \langle M \rangle$ , 并同时以下两项计算:

1. 逐步运行  $M(0)$ ; 若它停机, 则输出 1;
2. 枚举所有  $p \in \{0, 1\}^*$  并计算  $V(n_M, p)$ ; 若某次得到 1, 则输出 0。

**Pseudocode: Halting decider  $D$  from a complete proof system (input  $m$ )**

1. If  $m$  is not a valid Turing-machine encoding, output 0; otherwise, decode it as  $M$ .
2. Dovetail two branches: simulate  $M(0)$  step by step, and enumerate  $p \in \{0, 1\}^*$  while computing  $V(n_M, p)$ .
3. If the first branch finds that  $M(0)$  halts, output 1; if the second finds  $V(n_M, p) = 1$ , output 0.

若  $M(0)$  停机, 第一项计算会结束; 若它不停机, 则  $n_M \in \mathcal{T}$ , 完备性保证第二项最终找到证明。可靠性保证两项不会给出冲突的答案。因此这个过程计算  $\text{HALT}_0$ , 与定理 7.4 矛盾。  $\square$

这个结论给出了哥德尔不完备现象的计算核心。为了把它落实到算术语句, 下面固定一种具体的形式语言。

### 7.7.1 量化整数语句

**定义 7.5** (量化整数语句). 量化整数语句 (quantified integer statement, QIS) 是由下列成分组成的闭公式:

- 非负整数常数、变量以及运算  $+$ ,  $\times$  和关系  $=$ ;
- 逻辑联结词  $\wedge, \vee, \neg$ ;

- 量词  $\forall, \exists$ 。

本节约定量词的论域为  $\mathbb{N}$ 。闭公式没有自由变量，因而在标准自然数结构中具有确定的真值。

对非法编码约定结果为 0，并定义

$$\text{EVAL}(\varphi) = \begin{cases} 1, & \varphi \text{ 是真的 QIS,} \\ 0, & \varphi \text{ 是假的 QIS 或不是合法 QIS.} \end{cases}$$

**引理 7.1** (计算历史的算术化). 存在一个可计算变换, 把任意图灵机编码  $\langle M \rangle$  映射为 QIS  $\varphi_M$ , 使得

$$\varphi_M \text{ 为真} \iff M \text{ 在输入 } 0 \text{ 上停机.}$$

证明思路. 用自然数编码状态、纸带符号和有限格局, 再用一个自然数编码有限格局序列。“后一格局由前一格局经一条转移得到”是有限的算术条件; 序列取位、长度和相邻关系也可用加法、乘法和量词表达。因此可以有效写出一个公式, 断言“存在一个从  $M$  的初始格局开始、以停机格局结束的有限合法计算历史”。□

**定理 7.8** (QIS 真值不可判定). 函数 EVAL 不可计算。

证明. 若 EVAL 可计算, 则由 [引理 7.1](#) 可得

**Pseudocode: Halting decider  $D_0$  using EVAL (input  $m$ )**

1. If  $m$  is not a valid Turing-machine encoding, output 0.
2. Use the arithmetization lemma to compute  $\varphi_{M_m}$ .
3. Output  $\text{EVAL}(\varphi_{M_m})$ .

$$\text{HALT}_0(\langle M \rangle) = \text{EVAL}(\varphi_M),$$

这与 [定理 7.4](#) 矛盾。□

**定理 7.9** (量化整数真理的不完备性). 所有真 QIS 构成的集合没有可靠且完备的有效证明系统。

证明. 假设存在这样的验证器  $V$ 。给定合法 QIS  $\varphi$ , 同时枚举  $\varphi$  与  $\neg\varphi$  的所有候选证明, 并用  $V$  验证。二者恰有一个为真; 完备性保证其证明最终被找到, 可靠性保证假命题不会被证明。因此我们可以判定  $\varphi$  的真值, 这与 [定理 7.8](#) 矛盾。

**Pseudocode: QIS decider  $D_{\text{QIS}}$  from a complete proof system (input  $\varphi$ )**

1. If  $\varphi$  is not a valid QIS, output 0.
2. Dovetail candidate proofs  $p$  for  $\varphi$  and  $\neg\varphi$ , evaluating  $V(\varphi, p)$  and  $V(\neg\varphi, p)$ , respectively.
3. If a proof of  $\varphi$  is found, output 1; if a proof of  $\neg\varphi$  is found, output 0.

